

Spécifications Fonctionnelles : Projet Smart Campus

Table des matières

I. Contexte et Objectifs du Projet	2
1.1. Présentation générale	2
1.2. Établissement cible et public prioritaire	2
II. Architecture Technique et Contraintes	2
III. Rôles et Gestion des Utilisateurs	2
IV. Spécifications Fonctionnelles Détaillées & Périmètre du Projet	3
4.1. Périmètre Principal	3
4.2. Périmètre Avancé	4
V. Cas d'utilisation	4
VI. Structure des Données	5
VII. Règles Métier Académiques	5
7.1. Règles prioritaires (Garanties pour la démonstration)	5
7.2. Règles optionnelles	5
VIII. répartition des tâches	6

I. Contexte et Objectifs du Projet

1.1. Présentation générale

Dans le cadre du Projet piscine, ce document présente les spécifications fonctionnelles de SmartCampus, une plateforme web permettant de gérer les données académiques. L'objectif est de proposer un système cohérent modélisant les interactions réelles entre les étudiants, les enseignants et l'administration au sein d'un établissement supérieur.

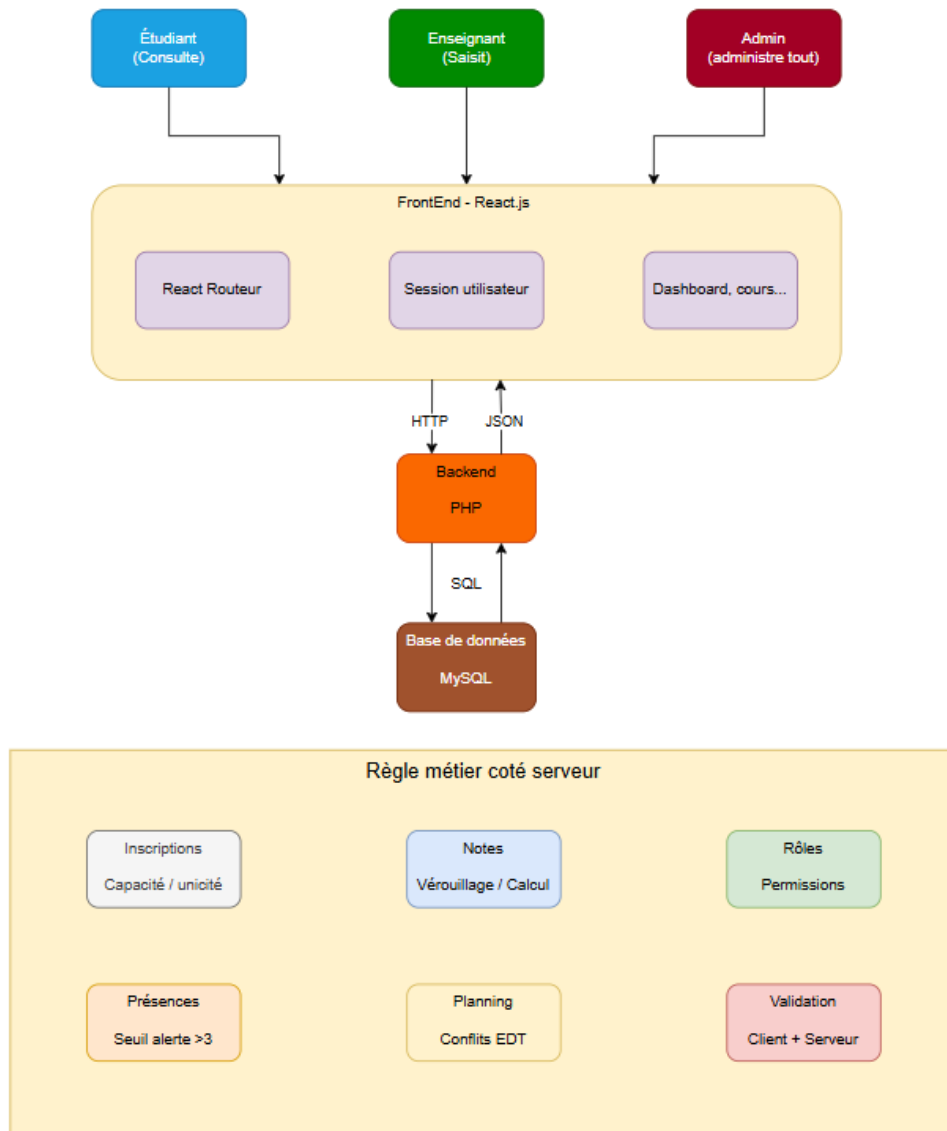
1.2. Établissement cible et public prioritaire

- Établissement cible : École d'ingénieurs spécialisée dans les technologies du numérique.
- Public prioritaire : Étudiants du cycle ingénieur, enseignants et administration.
- Vision du système : Une interface moderne, fluide, adaptée aux téléphones et aux ordinateurs, facilitant la circulation sécurisée des données.

II. Architecture Technique et Contraintes

L'application repose sur une architecture client-serveur stricte avec une séparation claire entre le Frontend et le Backend afin de garantir la maintenabilité du code.

- Frontend : HTML, CSS, JavaScript, et React pour une interface utilisateur dynamique et fluide.
- Backend : PHP pour traiter la logique métier côté serveur.
- Base de données : PhpMyAdmin



III. Rôles et Gestion des Utilisateurs

La plateforme applique une restriction stricte des actions selon les rôles utilisateurs. Chaque profil accède à un espace privé et sécurisé :

- **Administrateur** : Possède les droits absolus sur le système. Il crée, modifie et supprime les comptes utilisateurs (étudiants, enseignants) et configure les cours.
- **Enseignant** : Responsable de ses modules d'enseignement. Il saisit et valide les notes, enregistre les présences et consulte son planning pédagogique.
- **Étudiant** : Profil en consultation principale. Il accède à son tableau de bord, consulte ses notes, ses absences et son emploi du temps.

IV. Spécifications Fonctionnelles Détaillées & Périmètre du Projet

4.1. Périmètre Principal

- F01 : Module d'authentification et Sessions : Formulaire de connexion unique (Email / Mot de passe) avec redirection selon le rôle détecté.
- F02 : Tableaux de bord différenciés :
 - Espace Étudiant : Affichage de la moyenne générale calculée, de la liste des cours suivis et des prochaines séances.
 - Espace Enseignant : Grille des cours pris en charge, nombre d'étudiants inscrits par module et raccourcis vers la saisie des notes.
 - Espace Administrateur : Indicateurs globaux (nombre total d'étudiants, d'enseignants et de cours actifs) et accès rapide aux modules de configuration.
- F03 : Gestion des Étudiants : Interface d'administration pour ajouter un nouvel étudiant, consulter son parcours, modifier ses informations et le supprimer sans incohérence en base de données.
- F04 : Gestion des Enseignants : Panel d'administration pour créer des profils enseignants, modifier leurs responsabilités et leur affecter un ou plusieurs cours.
- F05 : Catalogue et Gestion des Cours : Interface permettant à l'administrateur de créer, modifier et supprimer des fiches de cours.
- F06 : Inscriptions aux cours : Module permettant d'inscrire ou de désinscrire un étudiant à un cours spécifique. L'interface de l'administrateur et de l'étudiant affiche en temps réel la liste des inscrits.
- F07 : Saisie et Calcul Automatique des Notes : Tableau matriciel permettant à l'enseignant de saisir et modifier les notes par type d'évaluation. Le système exécute instantanément en tâche de fond le calcul de la moyenne pondérée de l'étudiant et de la classe.

- F08 : Consultation et Validation des Notes : Processus de clôture définitive d'un module par l'enseignant. Après validation, les notes basculent sur le dashboard de l'étudiant en mode lecture seule.
- F09 : Emploi du Temps Dynamique : Calendrier interactif affichant de manière lisible les cours programmés, les horaires, les salles et les enseignants concernés. L'administration dispose d'une interface pour modifier les horaires avec contrôle des chevauchements.

4.2. Périmètre Avancé

Afin d'optimiser le temps de développement, les modules suivants sont qualifiés de hors périmètre initial. Nous espérons un bon développement de ces modules mais nous aviserons en fonction du temps ainsi que de notre avancement le long de cette semaine.

- Messagerie et Notifications Internes : Boîte de réception permettant l'envoi de messages entre utilisateurs selon leur rôle et affichage de notifications système
- Relevé de Notes PDF : Script permettant à l'étudiant d'exporter son relevé de notes semestriel officiel au format PDF.

V. Cas d'utilisation

- Cas d'utilisation 1 : Saisie, calcul et verrouillage d'un relevé de notes

Acteur principal : Enseignant

Préconditions : L'enseignant s'est authentifié avec succès. Le cours sélectionné dispose d'étudiants inscrits.

Tout d'abord L'enseignant sélectionne l'onglet "Mes cours" et clique sur la matière ciblée. Ensuite L'application génère un tableau dynamique listant l'ensemble des étudiants rattachés. L'enseignant peut ainsi saisir les notes numériques. À chaque modification, le Frontend React intercepte la valeur et recalcule dynamiquement à l'écran la moyenne pondérée de l'étudiant sans rechargement global. Il suffira à l'enseignant clique sur le bouton "Valider et Clôturer les notes" pour sauvegarder les notes saisies. Une notification de confirmation s'affiche pour prévenir du caractère irréversible de l'action.

Résultat : Les inputs de saisie sont désactivés pour l'enseignant. Les notes calculées apparaissent instantanément sur le dashboard de l'étudiant.

- Cas d'utilisation 2 : Inscription d'un étudiant à un cours par l'administrateur

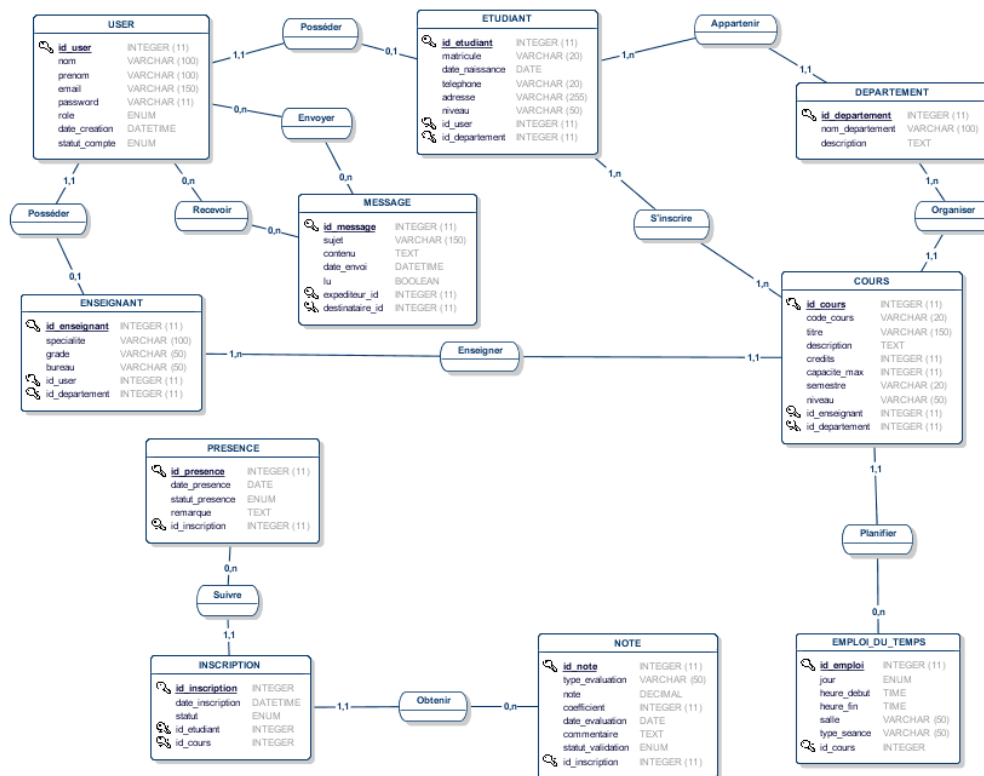
Acteur principal : Administrateur

Préconditions : L'administrateur est connecté. L'étudiant et le cours visés existent préalablement en base de données.

L'administrateur accède au module "Gestion des inscriptions". Il sélectionne l'étudiant concerné. Il cherche le cours souhaité puis clique sur le bouton « Inscrire l'étudiant ». Le système exécute en arrière-plan les requêtes de contrôle d'intégrité. Si les critères sont au remplis, le système insère la relation dans la base de données MySQL.

Résultat : L'étudiant est rattaché au cours. Le compteur de places disponibles est décrémenté et l'interface affiche un message de succès.

VI. Structure des Données



VII. Règles Métier Académiques

Le sujet propose 8 règles métier réalistes. Notre équipe s'engage à implémenter les 4 règles prioritaires suivantes au cœur de l'application, tandis que les 4 autres sont classées comme options de développement avancées :

7.1. Règles prioritaires (Garanties pour la démonstration)

- 1- Impossibilité de double inscription à un même cours : Un étudiant ne peut en aucun cas être inscrit deux fois à une même instance de cours.
- 2- Contrôle des capacités maximales des classes : Le système bloque automatiquement les nouvelles inscriptions dès que la capacité limite d'un cours ou d'une salle est atteinte.
- 3- Détection et blocage des conflits d'emploi du temps : Il est impossible d'affecter un enseignant, un groupe d'étudiants ou une salle de classe sur deux créneaux horaires qui se chevauchent.
- 4- Restriction stricte des actions selon les rôles utilisateurs : Chaque utilisateur est strictement confiné à l'espace correspondant aux privilèges de son rôle (Étudiant, Enseignant, Administrateur).
- 5- Archivage des données académiques d'un semestre précédent : Permettre de clôturer un semestre pour ouvrir une nouvelle période universitaire vierge, tout en figeant et en conservant l'historique consultable des anciennes données.

7.2. Règles optionnelles

Si le calendrier du projet le permet, la règle suivante pourra être ajoutée dans un second temps :

Gestion des absences avec seuil d'alerte : Déclencher une alerte visuelle automatique (badge rouge) sur le dashboard de l'administrateur dès qu'un étudiant dépasse un quota de 3 absences injustifiées.

VIII. répartition des tâches

Diagramme de Gantt

SmartCampus — Tâches

Kanban Table Timeline Gantt (Fin-31/05) Calendrier (deadlines) Dashboard projet

>> mai 2026

juin

17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21

Phase 1 — Conception

- Rendu Livrable 1 — 28/05 — 23h55 — À faire — PO — Critique — Phase 1 — Conception — 0 % — Equipe — 28/05/2026 23:55
- Création GitHub (repo + conventions) — Terminé — PO — Critique — Phase 1 — Conception — 10000 % — Tanguy — 27/05/2026
- Architecture système (React / PHP API / MySQL) — Terminé — PO — Critique — Phase 1 — Conception — 10000 % — Lucien — 27/0
- PowerPoint préliminaire (Livrable 1) — En cours — PO — Critique — Phase 1 — Conception — 7000 % — Equipe — 28/05/2026
- Répartition du travail (sans noms) — À faire — P1 — Haute — Phase 1 — Conception — 0 % — 28/05/2026
- MCD (modèle conceptuel) — Terminé — PO — Critique — Phase 1 — Conception — 10000 % — Tanguy — 27/05/2026
- MLD (modèle logique) — Terminé — PO — Critique — Phase 1 — Conception — 10000 % — Tanguy — 27/05/2026
- Storyboard (parcours démo) — Terminé — P1 — Haute — Phase 1 — Conception — 10000 % — Henrique — 27/05/2026
- Wireframes — Terminé — P1 — Haute — Phase 1 — Conception — 10000 % — Henrique — 27/05/2026
- Spécifications fonctionnelles — Terminé — PO — Critique — Phase 1 — Conception — 10000 % — Lucien — 27/05/2026
- Cahier des charges — En cours — PO — Critique — Phase 1 — Conception — 6000 % — 28/05/2026
- Analyse sujet — Terminé — PO — Critique — Phase 1 — Conception — 10000 % — 25/05/2026

+ Nouvelle page

Phase 2 — Frontend

- Dashboards UI (étudiant/enseignant/admin) — À faire — PO — Critique — Phase 2 — Frontend — Auth frontend (guards + s
- Gestion cours (UI React) — À faire — P1 — Haute — Phase 2 — Frontend — Auth frontend (guards + stockage session) — 0 %
- Gestion enseignants (UI React) — À faire — P2 — Moyenne — Phase 2 — Frontend — 0 % — Henrique — 31/05/2026
- Gestion étudiants (UI React) — À faire — P1 — Haute — Phase 2 — Frontend — Auth frontend (guards + stockage session) — 0 %
- Auth frontend (guards + stockage session) — À faire — PO — Critique — Phase 2 — Frontend — Auth backend (sessions + l
- Layout global (header/sidebar) — À faire — P1 — Haute — Phase 2 — Frontend — 0 % — 30/05/2026
- Pages authentication (login/logout) — À faire — PO — Critique — Phase 2 — Frontend — 0 % — 30/05/2026
- Routing (React Router) — À faire — P1 — Haute — Phase 2 — Frontend — 0 % — 28/05/2026
- Setup React (Vite/CRA) + structure — Terminé — PO — Critique — Phase 2 — Frontend — 10000 % — Alexandre — 27/05/2026

+ Nouvelle page

Phase 3 — Backend/API

- API inscriptions + API notes — À faire — PO — Critique — Phase 3 — Backend/API — Auth backend (sessions + login API) — 0 %
- CRUD cours (API) — À faire — PO — Critique — Phase 3 — Backend/API — Auth backend (sessions + login API) — Schéma
- CRUD enseignants (API) — À faire — P1 — Haute — Phase 3 — Backend/API — 0 % — Lucien — 31/05/2026
- CRUD étudiants (API + validations) — À faire — PO — Critique — Phase 3 — Backend/API — Auth backend (sessions + login
- Auth backend (sessions + login API) — À faire — PO — Critique — Phase 3 — Backend/API — 0 % — Lucien — 30/05/2026
- Connexion MySQL (PDO + env) — Terminé — PO — Critique — Phase 3 — Backend/API — 10000 % — Alexandre — 27/05/2026
- Architecture PHP API (routes + controllers) — Terminé — PO — Critique — Phase 3 — Backend/API — 10000 % — Alexandre — 27/05

+ Nouvelle page

Phase 4 — Base de données

- Schéma MySQL (tables + relations + contraintes) — En cours — PO — Critique — Phase 4 — Base de données — 4000 % — Alexan

+ Nouvelle page

Phase 5 — Métier

- Règles métier (double inscription, capacités, conflits horaires) — À faire — PO — Critique — Phase 5 — Métier — Schéma My

+ Nouvelle page

Phase 6 — Finalisation

- Tests + debugging + intégration finale — À faire — PO — Critique — Phase 6 — Finalisation — Dashboards UI (étudiant/ense
- README + packaging ZIP (install + comptes test) — À faire — PO — Critique — Phase 6 — Finalisation — Tests + debugginq
- Responsive design (global + pages clés) — À faire — P1 — Haute — Phase 6 — Finalisation — 0 % — Henrique — 31/05/2026
- Rendu Projet Final + Démo — 31/05 — 23h55 — À faire — PO — Critique — Phase 6 — Finalisation — 0 % — Equipe — 31/05/20
- Démo + PowerPoint final (Livrable final) — À faire — PO — Critique — Phase 6 — Finalisation — 0 % — 31/05/2026

+ Nouvelle page

+ Sens « Phase »

+ Nouvelle page

+ Nouveau groupe